

# CMOS Implementation and Analysis of an Optimized 4-Bit Carry Lookahead Adder

Prasanna N. R

Roll No: 2024102056

International Institute of Information Technology, Hyderabad

Email: prasanna.r@students.iiit.ac.in

## INDEX

- I. Introduction
- II. Proposed Structure for the CLA
- III. Design Details
- IV. Timing Constraints of Flip-Flop
- V. Stick Diagrams
- VI. Magic Layout of Building Blocks and the simulations
- VII. PreLayout Simulation in NGSpice
- VIII. FloorPlan For Magic Layout
- IX. Post-Layout Results
- X. Icarus Verilog HDL and Simulation
- XI. Acknowledgment

**Abstract**—This work presents the complete design flow of a 4-bit Carry Lookahead Adder (CLA), starting from logic design and circuit-level optimisation to physical layout and hardware implementation. The CLA provides a significant improvement over the conventional Ripple Carry Adder (RCA) by computing the carry-in for each bit using only the propagate ( $P_i$ ) and generate ( $G_i$ ) signals, instead of relying on a sequentially propagated carry. Such architectures are crucial in modern VLSI systems where 32-, 64- or 128-bit additions are common. We first derive an implementation-friendly form of the CLA equations, then design and simulate the circuit in NG-Spice, implement the layout in Magic, describe the architecture using Icarus Verilog HDL, and finally realise it on an FPGA to verify functionality on an oscilloscope. The main objectives are to minimise delay, reduce power consumption and optimise layout area.

**Index Terms**—Carry Lookahead Adder (CLA), CMOS VLSI, TSPC flip-flop, NG-Spice, Magic layout, FPGA, Icarus Verilog.

## I. INTRODUCTION

High-speed addition is a fundamental operation in arithmetic logic units (ALUs), digital signal processors, and general-purpose processors. While the Ripple Carry Adder (RCA) is simple and area-efficient, its linear delay with word length makes it unsuitable for high-performance systems. The Carry Lookahead Adder (CLA) alleviates this bottleneck by computing carries in parallel from propagate and generate signals.

In this work, we design a 5-bit CLA optimised at several levels: logic restructuring to reduce gate fan-in, circuit-level choices such as using pass-transistor logic (PTL) for XOR gates, transistor sizing for balanced rise/fall times, and architectural choices such as using TSPC flip-flops for pipelining. We implement and characterise the design both pre-layout and post-layout, and validate its operation on FPGA hardware.

## II. PROPOSED STRUCTURE FOR THE CLA

Fig. 1 shows the given high-level block diagram of the CLA. The system uses 16 flip-flops to register the inputs and outputs at the clock edge. The combinational CLA core consists of three sub-blocks: the P/G block, the carry generation block, and the sum block. The specification requires that the output be produced at the clock edge following the one at which the inputs are captured.

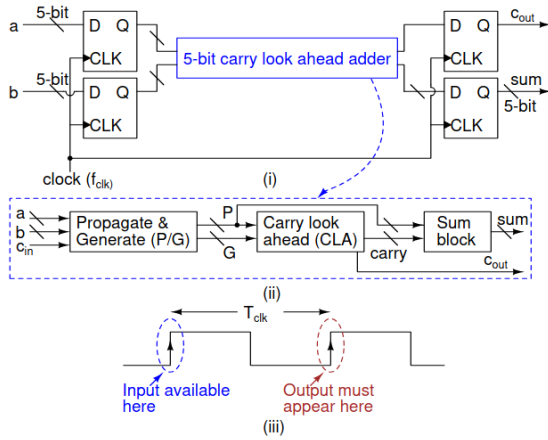


Fig. 1: High-level overview of the CLA design.

The conventional 5-bit CLA equations using propagate ( $P_i$ ) and generate ( $G_i$ ) signals and the input carry  $C_0$  are

$$C_1 = G_0 + P_0 C_0, \quad (1)$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0, \quad (2)$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0, \quad (3)$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0. \quad (4)$$

$$C_5 = G_4 + P_4 G_3 + P_4 P_3 G_2 + P_4 P_3 P_2 G_1 + P_4 P_3 P_2 P_1 G_0 + P_4 P_3 P_2 P_1 P_0 C_0. \quad (5)$$

Fig. 2 shows the complete implementation of the proposed carry generation and sum computation circuitry. Given the propagate and generate signals  $P_i = A_i \oplus B_i$  and  $G_i = A_i B_i$ , the circuit realises all carry expressions using only two-input or three-input AND and OR gates, as well as XOR gates for the sum logic. Each block in the figure corresponds to one of the terms in the carry equations, and their interconnection forms the full carry lookahead network required for the adder. Thus, Fig. 2 represents the entire combinational CLA required to compute all carry signals and sum outputs from the input bits.

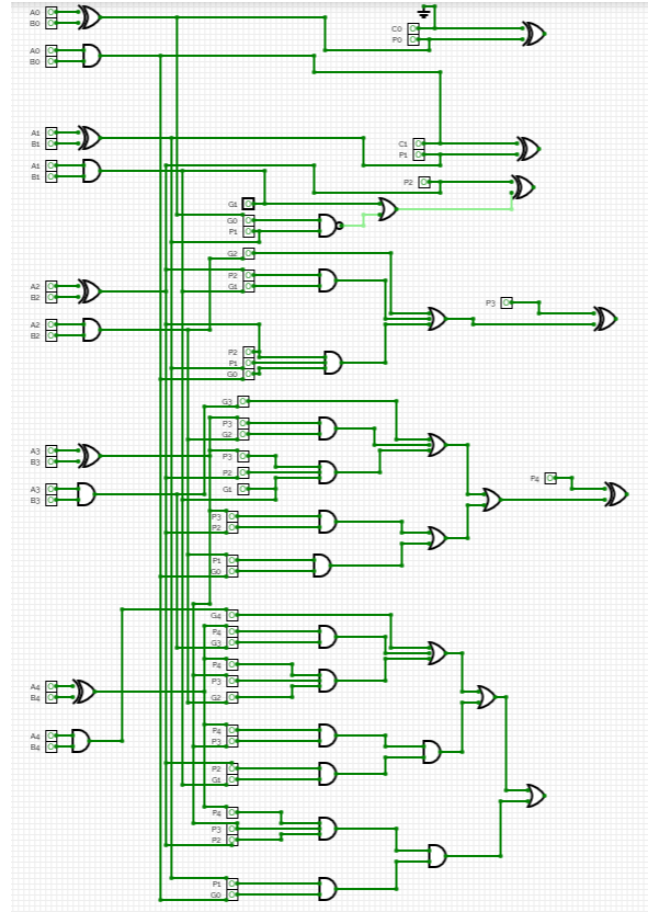


Fig. 2: Front-end overview of the CLA design.

### III. DESIGN DETAILS

We use a mixed topology combining standard CMOS logic and pass-transistor logic (PTL). Basic gates (NOT, NAND, NOR, AND, OR) are implemented using CMOS, while XOR gates are realised using PTL for better area and transistor count.

D-type flip-flops are implemented using the TSPC (True Single-Phase Clock) topology due to its superior properties such as near-zero hold time and suitability for high-speed designs.

#### A. Transistor Sizing

We first define the minimum inverter as reference:

$$W_n = 10\lambda, \quad W_p = 2W_n = 20\lambda.$$

All other gates are sized to approximately match this inverter's effective resistance in order to balance rise and fall times while maintaining reasonable driving capability and acceptable layout area.

Table I summarises the sizing used for the basic gates.

#### B. TSPC Flip-Flop

The positive edge-triggered TSPC flip-flop used in the design is shown in Fig. 3. The topology consists of two latches controlled by a single-phase clock and is optimised for high-speed operation.

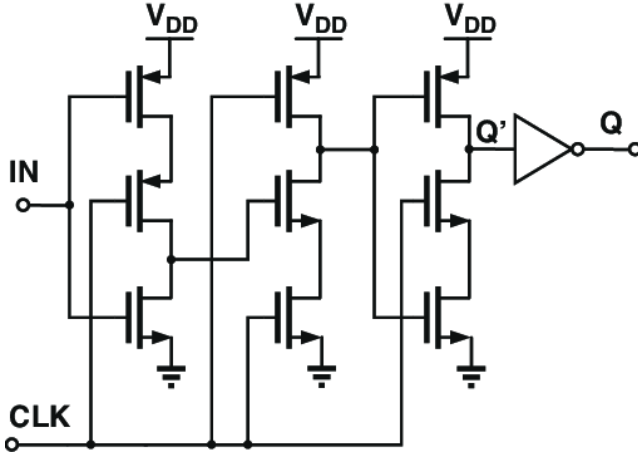


Fig. 3: TSPC flip flop.

The XOR gate is implemented using PTL style (takes 4 transistors of passing logic and 2 inverters), which consumes lesser area than that of the standard CMOS implementation.

TABLE I: Transistor sizing of basic gates

| Gates        | $W_n$       | $W_p$       |
|--------------|-------------|-------------|
| NOT          | $10\lambda$ | $20\lambda$ |
| 2-input NAND | $20\lambda$ | $20\lambda$ |
| 2-input NOR  | $10\lambda$ | $40\lambda$ |
| 2-input AND  | $20\lambda$ | $20\lambda$ |
| 2-input OR   | $10\lambda$ | $40\lambda$ |
| 3-input NAND | $30\lambda$ | $20\lambda$ |
| 3-input NOR  | $10\lambda$ | $60\lambda$ |

### IV. TIMING CONSTRAINTS OF FLIP-FLOP

To characterise the TSPC flip-flop, we evaluate its setup time, hold time and clock-to-Q delay  $t_{CQ}$  using NG-Spice transient simulations. Fig. 4 shows typical waveforms of the clock, data input and flip-flop output.

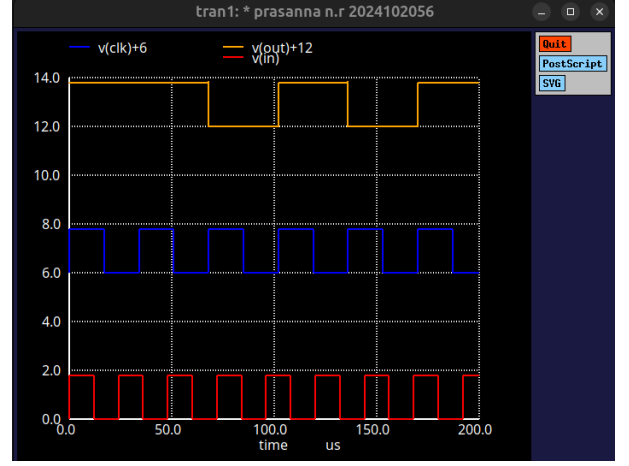


Fig. 4: TSPC flip-flop simulation waveforms.

The setup time is obtained as the minimum interval before the active clock edge for which data must be stable to guarantee correct latching. In this TSPC-based implementation, it equals the  $t_{CQ}$  of the preceding negative-edge latch, since the data node must settle sufficiently before the positive-edge latch samples it.

The hold time is effectively zero, since once the clock goes high, the negative latch turns off and the input to the positive latch is isolated from changes at the D input.

The clock-to-Q delay  $t_{CQ}$  is measured as the 50%–50% delay between the rising clock edge and the corresponding transition at the output. Fig.5 illustrates this measurement.

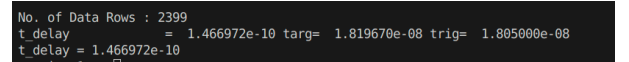


Fig. 5: Time delay between Clock and Q.

From simulation,

$$\text{setup time} \approx 20 \text{ ps},$$

$$\text{hold time} \approx 0 \text{ ps},$$

$$t_{CQ} \approx 146.67 \text{ ps}.$$

## V. STICK DIAGRAMS

Stick diagrams are drawn for all basic gates and the flip-flop to guide layout regularity and routing. The stick diagrams shows representative diagrams.

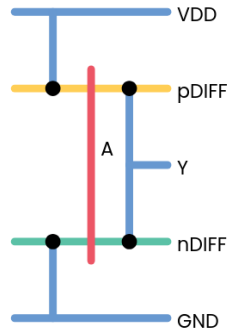


Fig. 6: Inverter gate.

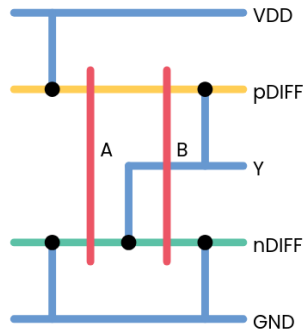


Fig. 7: NOR.

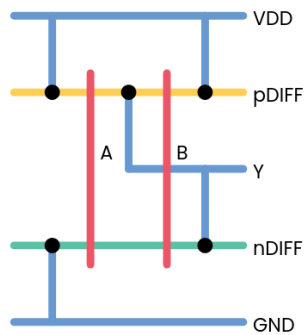


Fig. 8: NAND.

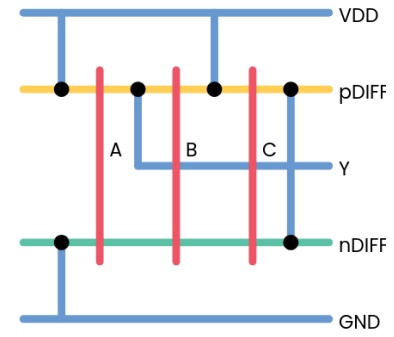


Fig. 9: 3 input NAND.

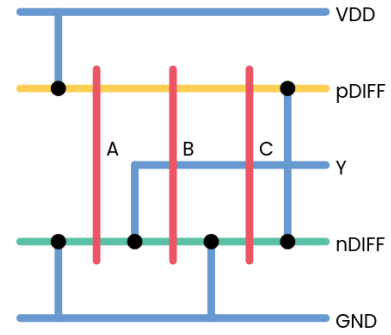


Fig. 10: 3 input NOR.

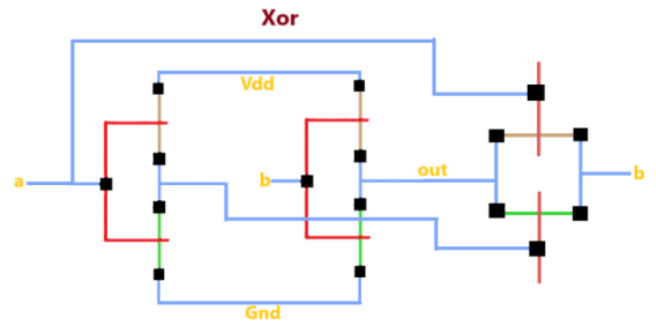


Fig. 11: PTL XOR.

## VI. MAGIC LAYOUT OF BUILDING BLOCKS AND THE SIMULATIONS

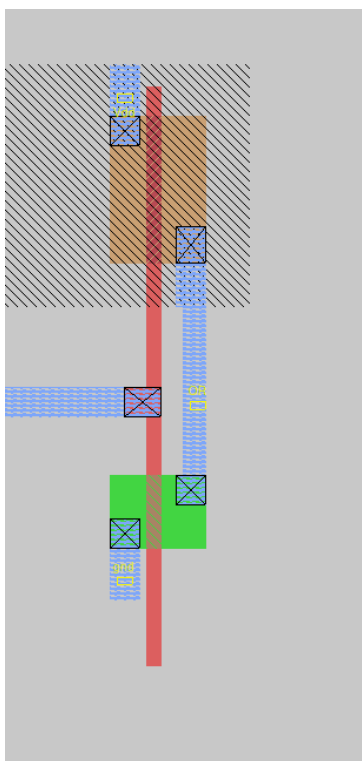


Fig. 12: Inverter gate.

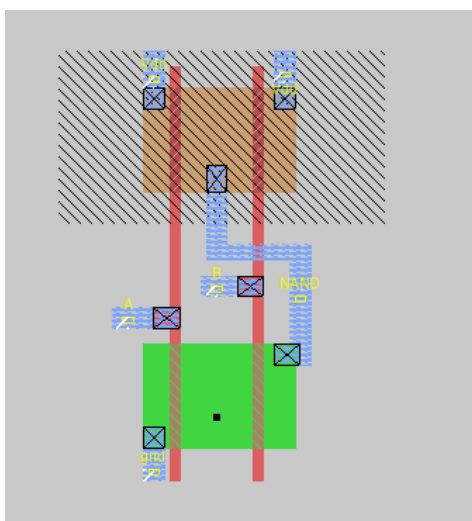


Fig. 13: Nand gate.

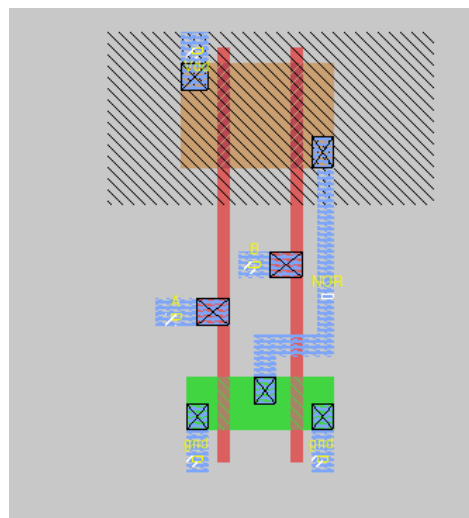


Fig. 14: Nor gate.

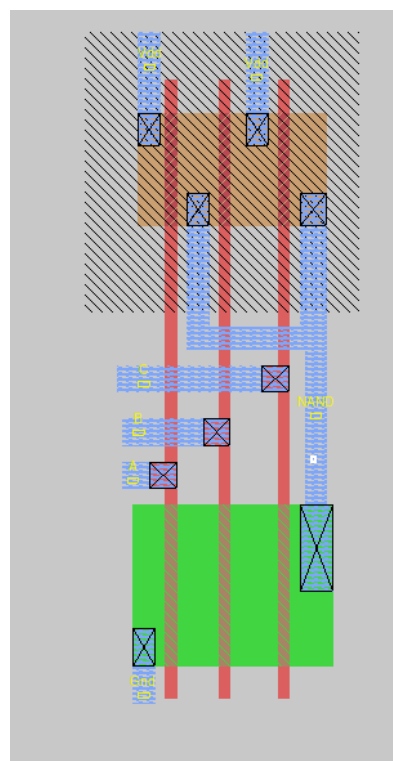


Fig. 15: 3-input NAND.

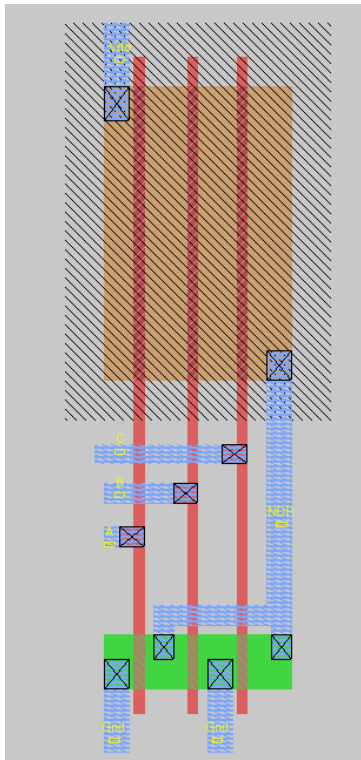


Fig. 16: 3-input NOR.

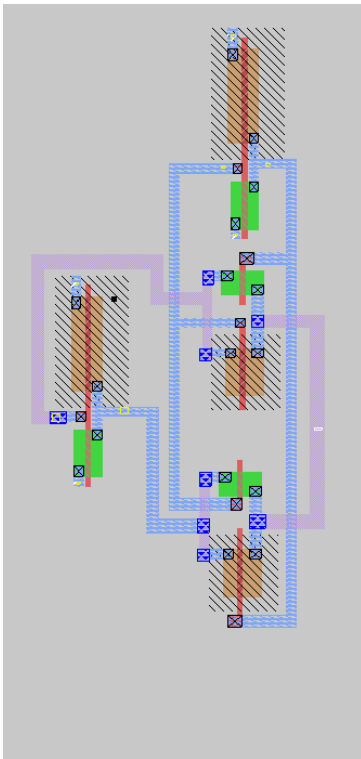


Fig. 17: XOR diagram.

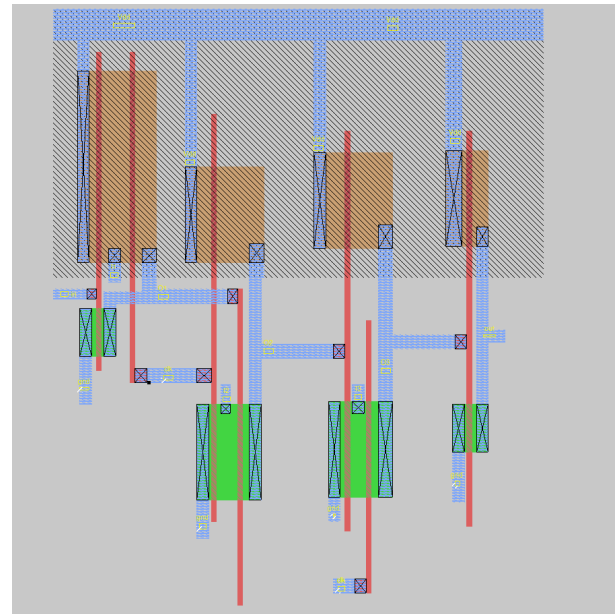


Fig. 18: D Flip Flop.

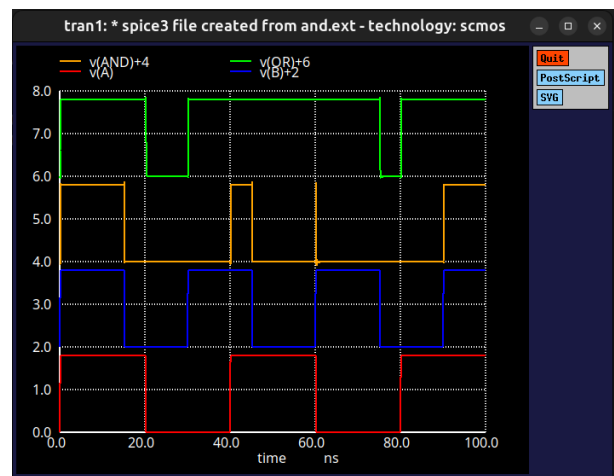


Fig. 19: AND OR gates simulations.

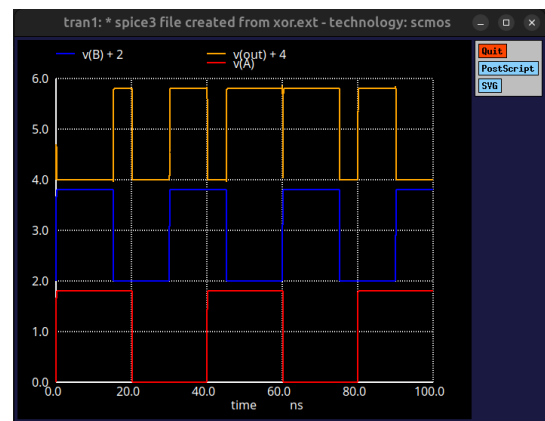


Fig. 20: XOR simulations.

## VII. PRE-LAYOUT SIMULATION IN NG-SPICE

After integrating the pre-layout components and thus we give a test case A = 10101 and B = 00011 we get

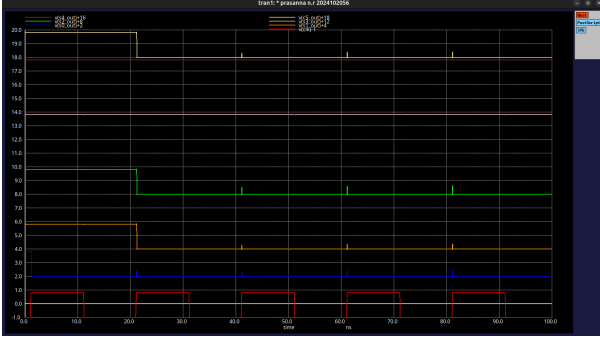


Fig. 21: S = 11000.

The corresponding input stimulus is shown in Fig.00. The longest delay occurs when the signal flows from b0 to C5

The worst case pre-layout =  $7.301341e-10$  The synchronous timing constraint for correct pipeline operation is

$$T_{clk} \geq T_{CQ} + T_{su} + T_{prop}.$$

Using the extracted simulation values,

$$T_{CQ} = 146.7 \text{ ps}, \quad T_{su} = 0.2 \times 10^{-10} \text{ s}, \quad T_{prop} = 7.3 \times 10^{-10} \text{ s}$$

and noting that

$$T_{su} = 0.2 \times 10^{-10} \text{ s} = 2 \times 10^{-11} \text{ s} = 20 \text{ ps},$$

the minimum allowable clock period becomes

$$T_{clk} \geq 146.7 \text{ ps} + 20 \text{ ps} + 730 \text{ ps} = 896.7 \text{ ps} = 0.8967 \text{ ns}.$$

In the implemented design, the clock period used is

$$T_{clk} = 0.90 \text{ ns},$$

which satisfies the timing requirement with a small positive margin. The corresponding operating frequency is the reciprocal of the clock period:

$$f_{clk} = \frac{1}{T_{clk}} = \frac{1}{0.90 \times 10^{-9}} \approx 1.11 \text{ GHz}.$$

Thus, the system operates safely at approximately

$$f_{clk} \approx 1.11 \text{ GHz}.$$

## VIII. FLOORPLAN FOR MAGIC LAYOUT

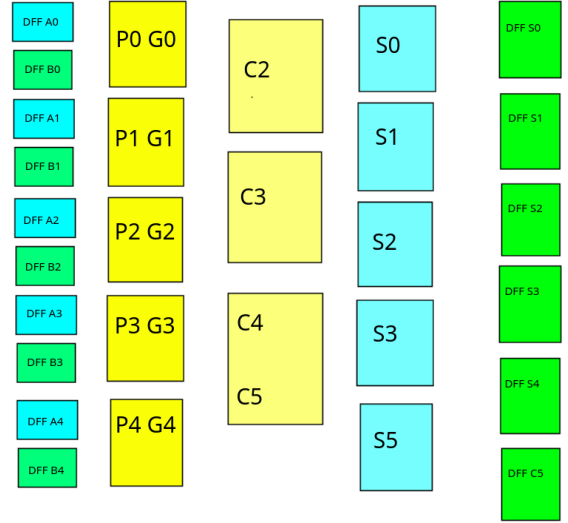


Fig. 22: Floor layout.

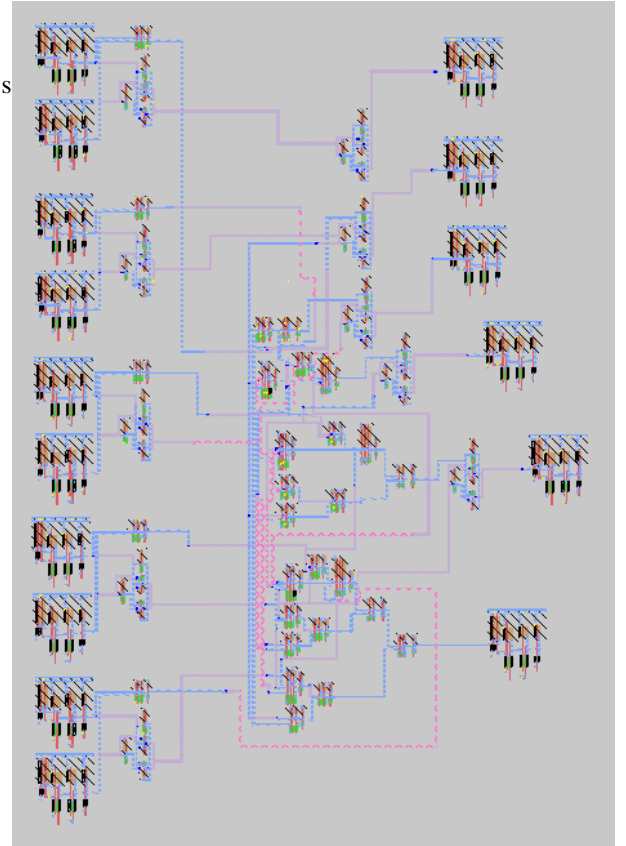


Fig. 23: Full Magic.

The full layout occupies an area of

$$1968\lambda \times 2834\lambda.$$

## IX. POST-LAYOUT RESULTS

Fig. 23 shows the final Magic layout of the entire CLA, including all logic and flip-flops. The overall layout dimensions are approximately  $1968\lambda \times 2834\lambda$ , with additional unused space at the corners.

```
No. of Data Rows : 2398
t_delay           = 1.477566e-10 targ= 1.819776e-08 trig= 1.805000e-08
t_delay           = 1.477566e-10
```

Fig. 24: Tcq of DFF post simulations.

```
Reference value : 9.12636e-08
No. of Data Rows : 1176
t_delay           = 7.734954e-10 targ= 1.823495e-09 trig= 1.050000e-09
```

Fig. 25: Propagation delay for Post layout.

The synchronous timing requirement for correct operation is given by

$$T_{clk} \geq T_{CQ} + T_{su} + T_{prop}.$$

From post-layout simulation, the extracted timing parameters are

$$T_{CQ} = 147.7 \text{ ps}, \quad T_{su} = 0.2 \times 10^{-10} \text{ s}, \quad T_{prop} = 773 \text{ ps}.$$

Since

$$T_{su} = 0.2 \times 10^{-10} \text{ s} = 2 \times 10^{-11} \text{ s} = 20 \text{ ps},$$

the minimum allowable clock period becomes

$$T_{clk} \geq 147.7 \text{ ps} + 20 \text{ ps} + 773 \text{ ps} = 940.7 \text{ ps} = 0.9407 \text{ ns}.$$

In the implemented design, the chosen clock period is

$$T_{clk} = 0.923 \text{ ns},$$

which is slightly smaller than the required minimum, indicating a timing violation. The corresponding operating frequency is

$$f_{clk} = \frac{1}{T_{clk}} = \frac{1}{0.923 \times 10^{-9}} \approx 1.083 \text{ GHz}.$$

TABLE II: Comparison of Pre-layout and Post-layout Timing Parameters

| Parameter                         | Pre-layout | Post-layout |
|-----------------------------------|------------|-------------|
| Clock-to-Q $T_{CQ}$ (ps)          | 146.7      | 147.7       |
| Setup time $T_{su}$ (ps)          | 20         | 20          |
| Propagation delay $T_{prop}$ (ps) | 730        | 773         |
| Required $T_{clk}$ (ns)           | 0.9        | 0.923       |
| Maximum frequency (GHz)           | 1.1        | 1.083       |

## X. ICARUS VERILOG HDL AND SIMULATION

A structured Verilog description of the 5bit CLA is written using the software Icarus Verilog.

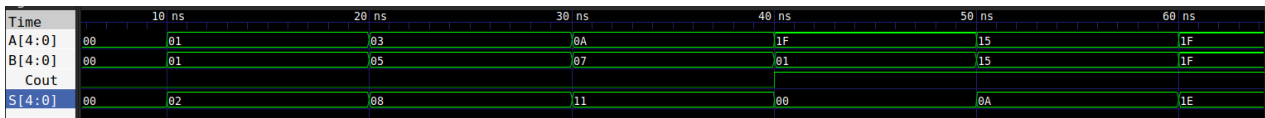


Fig. 26: TestCase.

## ACKNOWLEDGMENT

I would like to thank Dr. Abhishek Shrivastava for invaluable guidance throughout the project and making me interested in the course work. A special thanks to the TAs for conducting tutorials that enriched our foundation to the this course. I would like to thank my friends for many useful discussions and ideas.

## REFERENCES

1. Digital Logic and Computer Design by Morris Mano.
2. Verilog HDL: A Guide to Digital Design and Synthesis.
3. CMOS VLSI Design (fourth edition) by Weste and Harris.